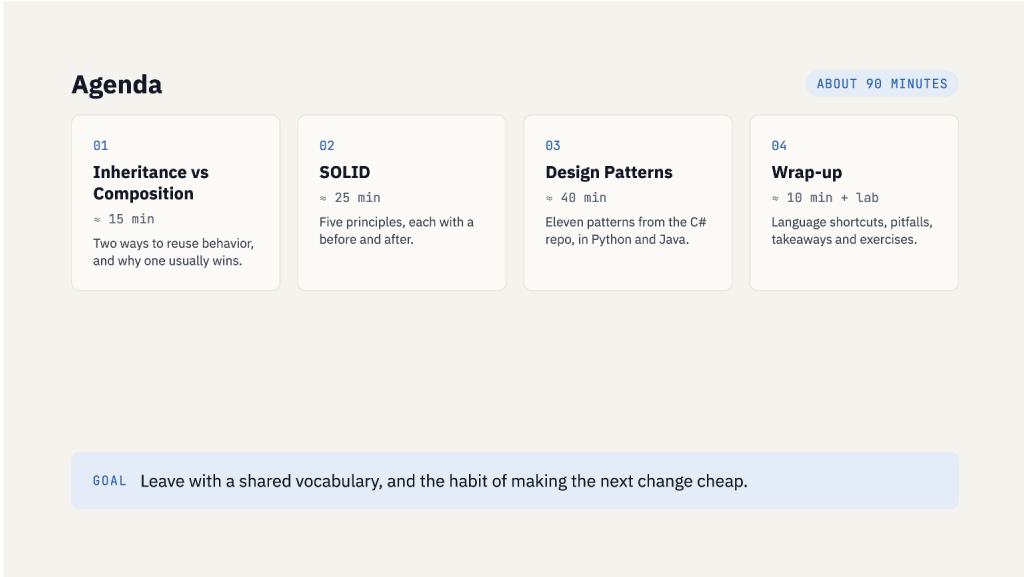
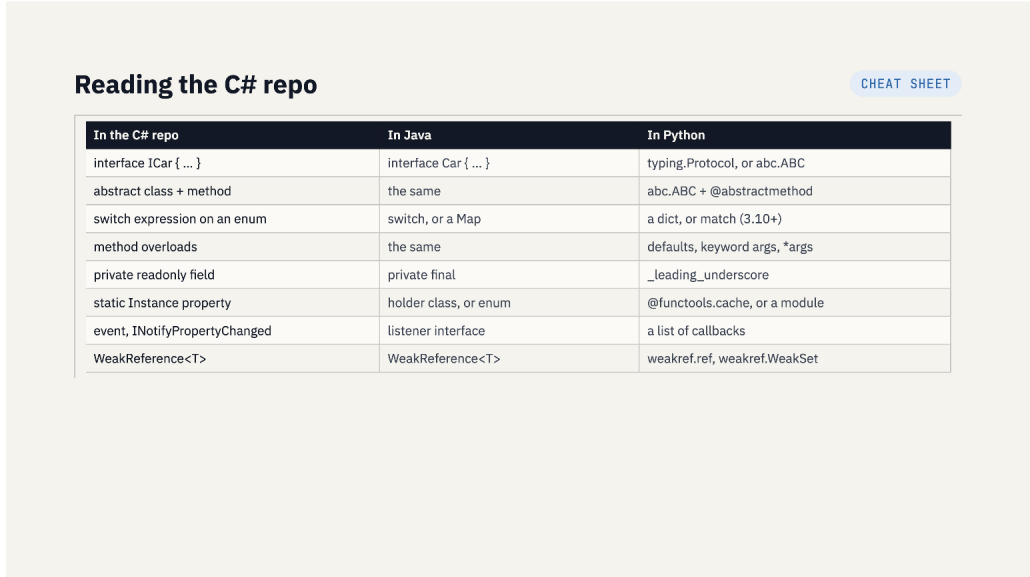
Welcome. Today is about one question: how do we write code that is cheap to change? We'll go in three steps. First, the most basic design choice in OO code: inherit or compose. Second, SOLID, five principles that all point at the same goal. Third, design patterns, the named, reusable solutions, using the examples from the design-patterns-c-sharp repo. Every code slide shows the Python on the left and the same example in Java on the right: the ideas don't depend on the language. All of the code runs; the Python has a pytest suite and the Java compiles and runs on Java 21.



Roughly 90 minutes plus an optional 30-minute lab at the end. Part 1 is short but it sets up everything else: almost every pattern later is either composition or a careful use of inheritance. Part 2 gives us the vocabulary to say why a design is good or bad. Part 3 is the catalogue. The code slides are dense because each shows two languages; you don’t need to read the Java line by line, it’s there to show the same shape. Feel free to interrupt.

The patterns and principles are language-independent: they are about who knows about whom. Python makes the ideas easy to see; Java, next to it, shows the same structure with every type spelled out, which is closer to the C# in the repo. The one caveat for Python: it won't stop you from breaking a contract at runtime, so every example here is also checked with mypy. In Java, javac does that job.

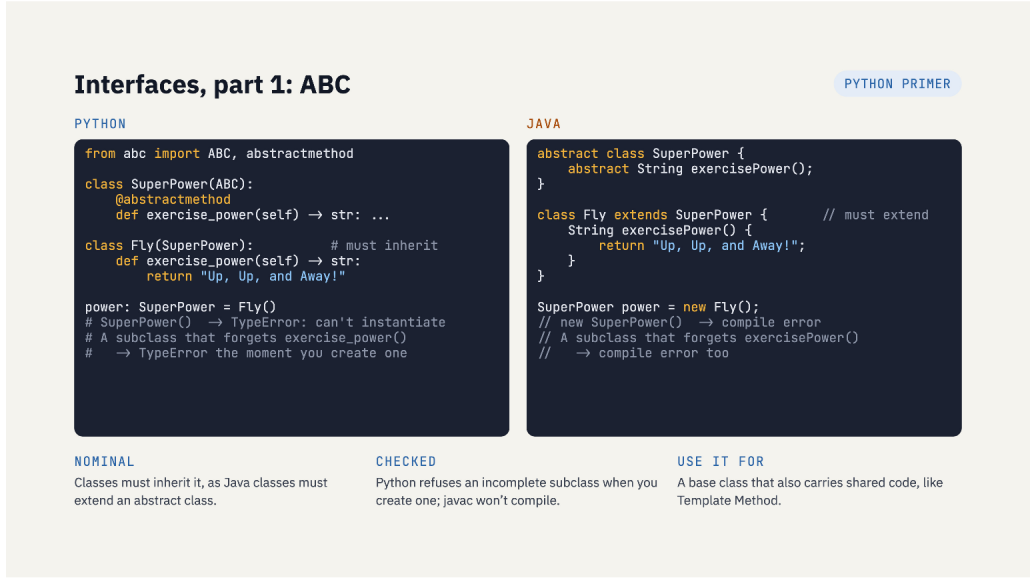


Reading the C# repo

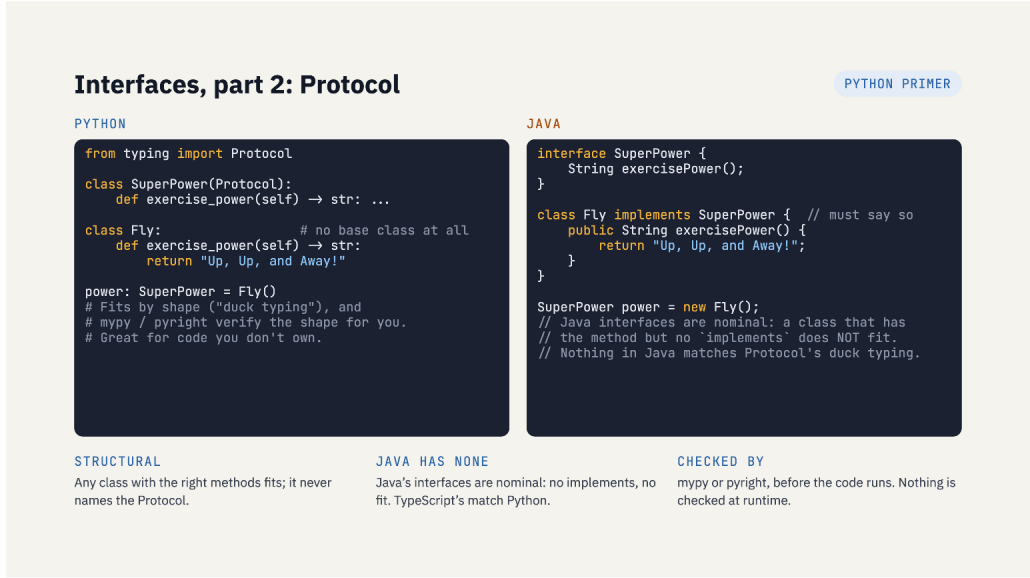
CHEAT SHEET

In the C# repo	In Java	In Python
interface ICar { ... }	interface Car { ... }	typing.Protocol, or abc.ABC
abstract class + method	the same	abc.ABC + @abstractmethod
switch expression on an enum	switch, or a Map	a dict, or match (3.10+)
method overloads	the same	defaults, keyword args, *args
private readonly field	private final	_leading_underscore
static Instance property	holder class, or enum	@functools.cache, or a module
event, INotifyPropertyChanged	listener interface	a list of callbacks
WeakReference<T>	WeakReference<T>	weakref.ref, weakref.WeakSet

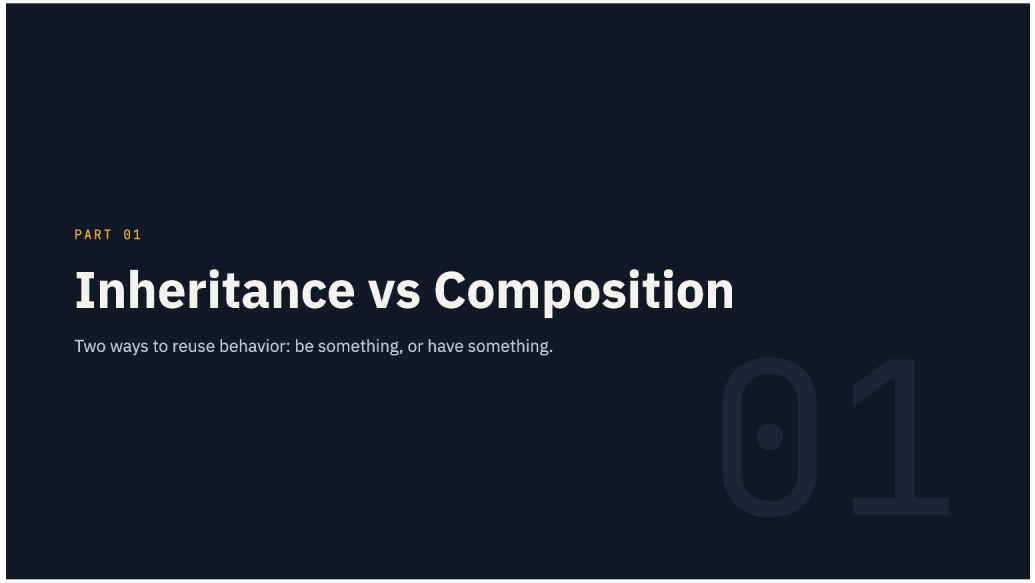
If you open the original repo, this is the translation table. C# and Java are close cousins, so the Java column is often ‘the same’. The big Python differences: interfaces become Protocols or ABCs, switch expressions become dict lookups because classes are callable, and overloads become default and keyword arguments. Privacy is by convention: one leading underscore means ‘not part of the interface’.



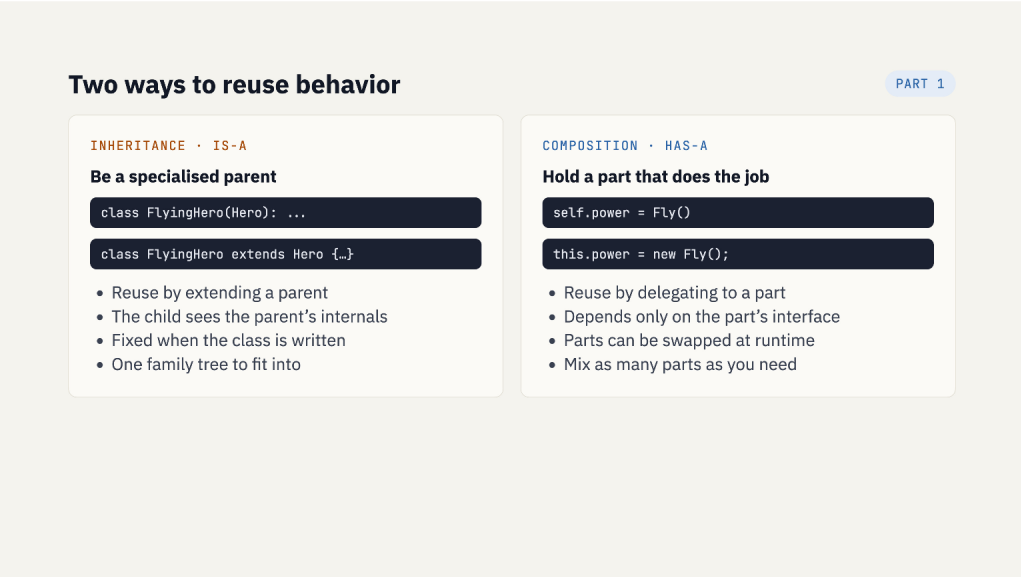
Both languages: SuperPower declares exercise_power, Fly inherits and implements it. Python's ABC refuses to instantiate the base, or any subclass that forgot a method, but only when you try. Java refuses at compile time. In this deck we use an ABC where the base class also carries code.



This is the one place where the languages really differ. In Python, Fly never mentions SuperPower, yet a type checker accepts it because the shape matches. Java has no structural interfaces: Fly must say implements. Protocols are great for code you don’t own. Most interfaces in the Python examples are Protocols; in Java they’re ordinary interfaces.



Part 1. Before any principle or pattern, there’s one choice we make every day: when I want to reuse some behavior, do I subclass, or do I hold an object that has the behavior?

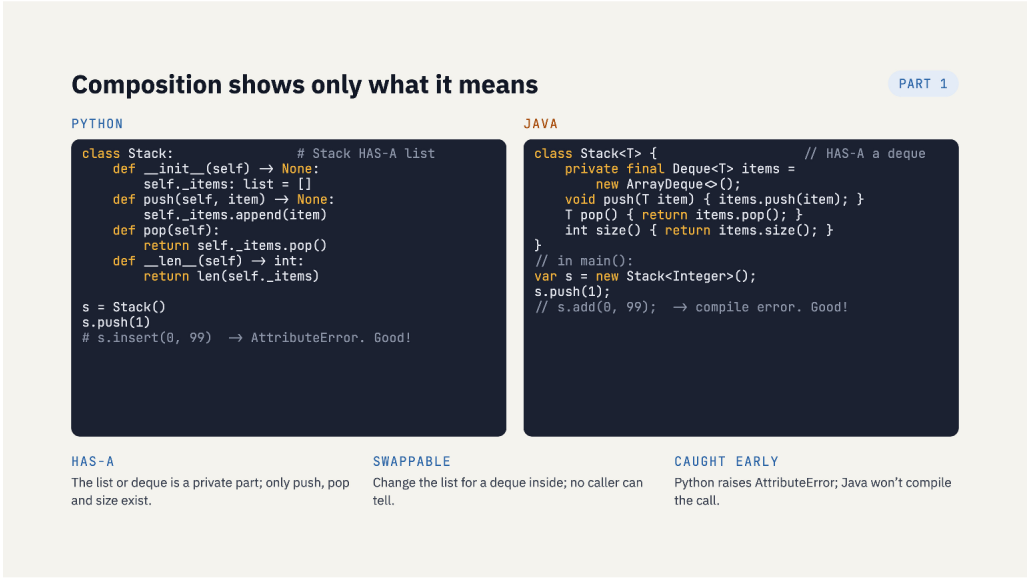


Inheritance says: I am a kind of my parent, and I get everything it has. Composition says: I have a part, and I ask it to do the work. Each card shows the Python and then the Java spelling. Read the bullet pairs across: extending versus delegating, seeing internals versus seeing only an interface, decided at class-definition time versus swappable at runtime, one tree versus any mix.

Let’s model heroes with inheritance. A flying hero, a fighting hero. Now we need a hero who does both. Python lets us write class FlyingFightingHero(FlyingHero, FightingHero), but act() only flies: the method resolution order picks FlyingHero first. Java won’t compile two superclasses at all, so you end up copying code. Either way, three behaviors give seven combinations: the class explosion.

Same requirement, composed. A Hero has a list of powers; each power is a small object with exercise(). A flying fighter is just Hero(Fly(), Fight()). Adding a swimming power is one new class, and every combination comes for free. In Java a one-method interface is a functional interface, so the powers can even be lambdas. We'll meet this shape again as the Strategy pattern.

A subtler problem: a subclass inherits the parent's whole public interface, whether it makes sense or not. Our Stack is still a list, so anyone can insert at index 0 or sort it. The JDK itself made this mistake: java.util.Stack extends Vector, which is why the Javadoc recommends ArrayDeque instead.



The composed Stack keeps its storage private and exposes only push, pop and a size. Its interface says exactly what it means, and we could swap the storage without anyone noticing. Notice the Java version uses ArrayDeque, the JDK's recommended stack.

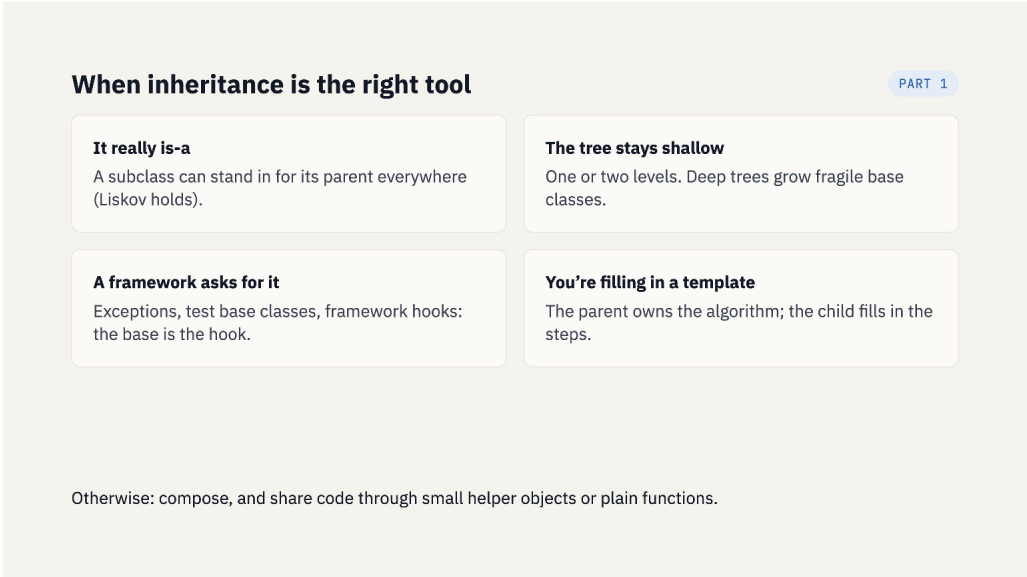
The trade-offs side by side

PART 1

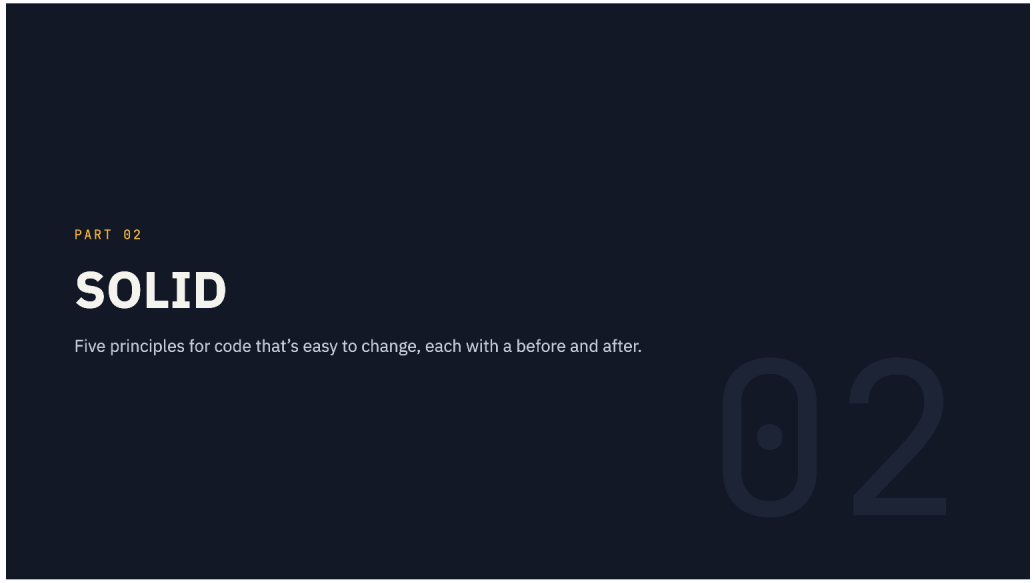
	Inheritance	Composition
Relationship	is-a	has-a, uses-a
Coupling	high: child knows parent internals	low: only the part's interface
Change behavior	when the class is written	at runtime: swap the part
Combine behaviors	a subclass per combination	mix parts freely
Testing	through the whole hierarchy	parts alone; inject fakes
Up-front cost	less code	a little more wiring

Here's the summary. Composition wins on coupling, flexibility and testability; inheritance wins on brevity, because you get the parent's code for free. That last row matters: composition isn't free, you write a bit of delegation. For a true, stable is-a relationship, inheritance is still the simpler tool.

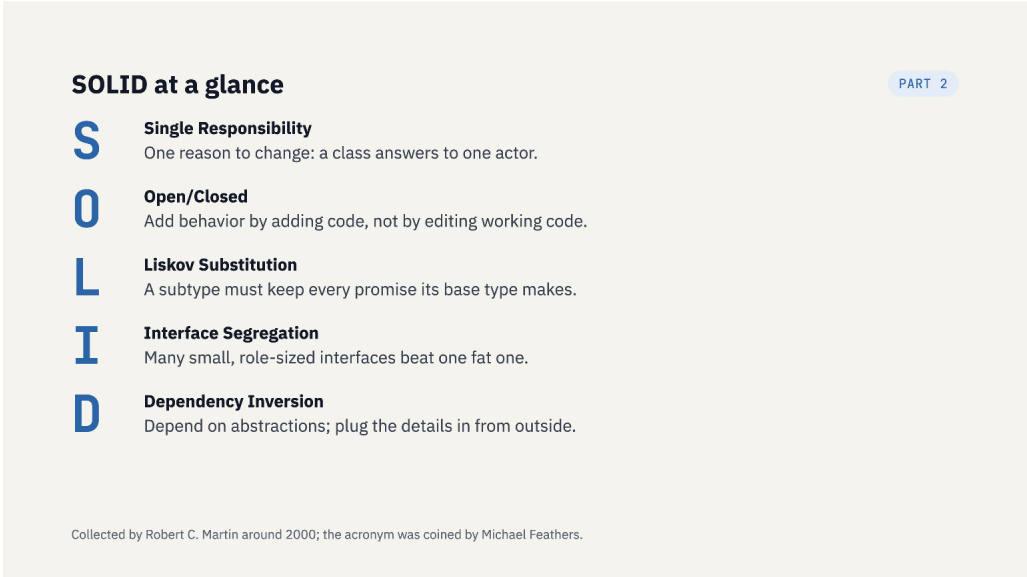
This is the most quoted line from the Gang of Four book, and it’s the thread through the rest of the talk. Note the word favor: it’s a default, not a ban. Next slide: when inheritance is the right call.



Four green lights for inheritance. One: the substitution test, which is the L in SOLID, coming up next. Two: shallow hierarchies; the deeper the tree, the more a change at the top breaks something at the bottom. Three: frameworks built around subclassing, like custom exceptions in both languages, unittest.TestCase in Python or an abstract Spring base class in Java. Four: Template Method.



Part 2: SOLID. Robert C. Martin collected these principles around 2000; Michael Feathers later arranged them into the acronym. Each one is a different angle on the same goal: when a requirement changes, the change should be small and local.



The five, one line each. Each principle gets two slides next: the ‘before’ and the ‘after’, both in Python and Java. Four of the five examples come straight from the repo’s own code.

One class serves three masters. Finance owns the total, the design team owns the HTML, ops owns where it’s saved. A change for any of them touches this class and risks the other two.

Each class now has exactly one reason to change. A useful test: describe the class without saying ‘and’. Another: who would ask for this change? If it’s different people, split it.

This works until marketing invents a new customer type every quarter. Each one means editing, re-testing and redeploying checkout, the riskiest function in the shop.

checkout now depends on the Discount abstraction; a new rule is a new class, or in Java a new lambda. The repo’s Decorator example is its best illustration of OCP: new car upgrades without touching BaseCar.

This one is in the repo itself. BadGuyAdapter claims to be an ICharacter, but its Flee throws, and the repo's own test asserts the exception. In Java we call the interface GameCharacter because java.lang.Character already exists. Barbara Liskov stated the principle in 1987.

Split the contract into Combatant and Escaper; only real escapers get passed to retreat. Now the mistake is caught before the code runs: by mypy in Python, by the compiler in Java. Both commented lines on the slide were checked: they really fail.

The repo's IIngredientFactory has five methods, including sausage and pepperoni. A vegetarian kitchen must still implement them, and the only honest implementation is to throw, which is an LSP violation waiting to happen.

I — Interface Segregation: after

SOLID 4/5

PYTHON

```python
class BaseIngredients(Protocol):
    def make_dough(self) -> str: ...
    def make_sauce(self) -> str: ...
    def make_cheese(self) -> str: ...

class MeatIngredients(Protocol):
    def make_sausage(self) -> str: ...
    def make_pepperoni(self) -> str: ...

class VeggieKitchen:          # only what it does
    def make_dough(self): return "wheat"
    def make_sauce(self): return "tomato"
    def make_cheese(self): return "vegan"

def cheese_pizza(k: BaseIngredients) -> list[str]:
    return [k.make_dough(), k.make_sauce(),
            k.make_cheese()]
```

JAVA

```java
interface BaseIngredients {
    String makeDough();
    String makeSauce();
    String makeCheese();
}
interface MeatIngredients {
    String makeSausage(); String makePepperoni();
}

class VeggieKitchen implements BaseIngredients {
    public String makeDough()  { return "wheat"; }
    public String makeSauce()  { return "tomato"; }
    public String makeCheese() { return "vegan"; }
}
static List<String> cheesePizza(BaseIngredients k) {
    return List.of(k.makeDough(), k.makeSauce(),
                   k.makeCheese());
}
```

THE FIX

Small, role-sized interfaces. A kitchen implements only what it does.

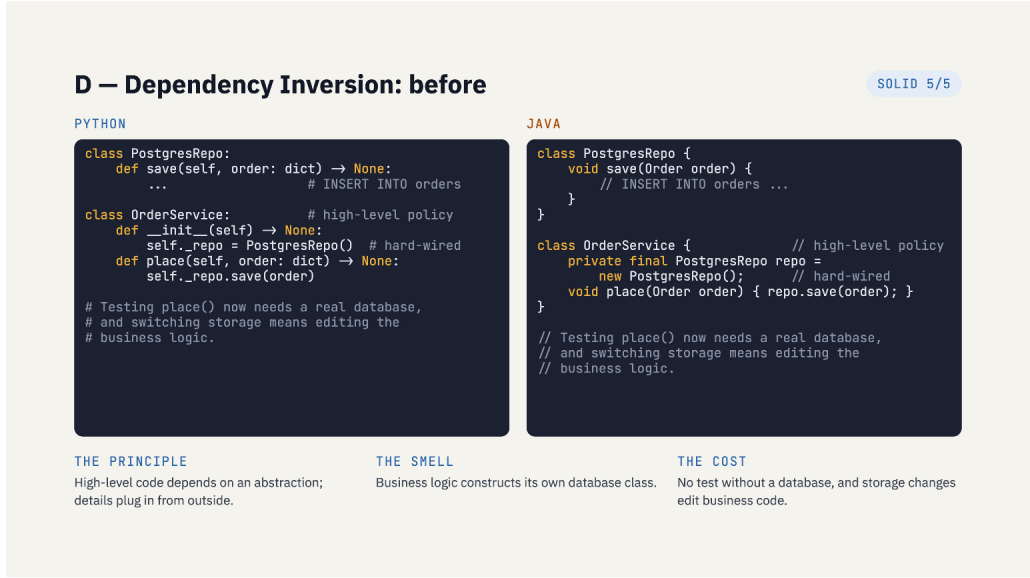
PYTHON

Protocols cost nothing to add: classes don't declare them.

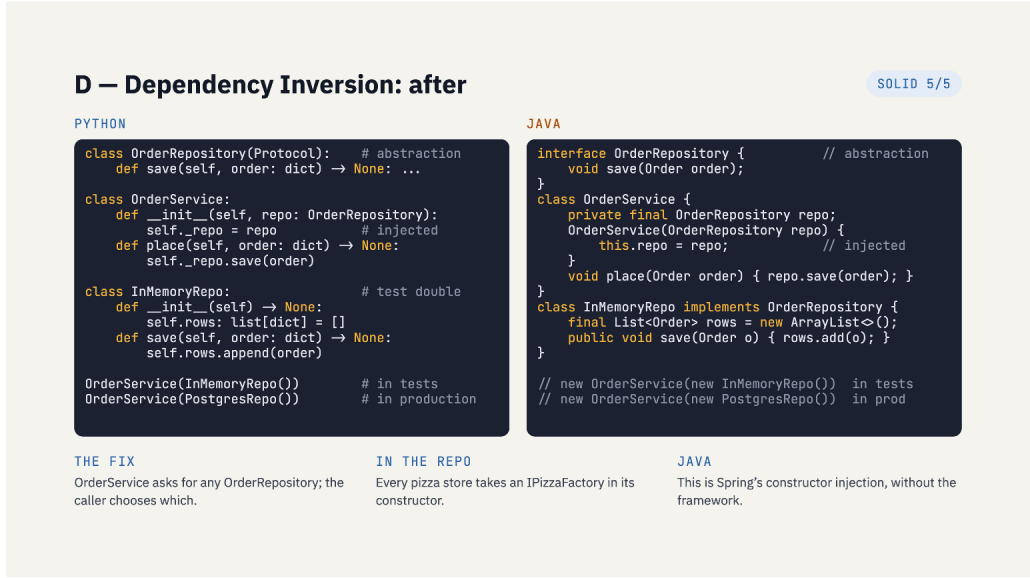
JAVA

A meat kitchen would implement both: implements A, B.

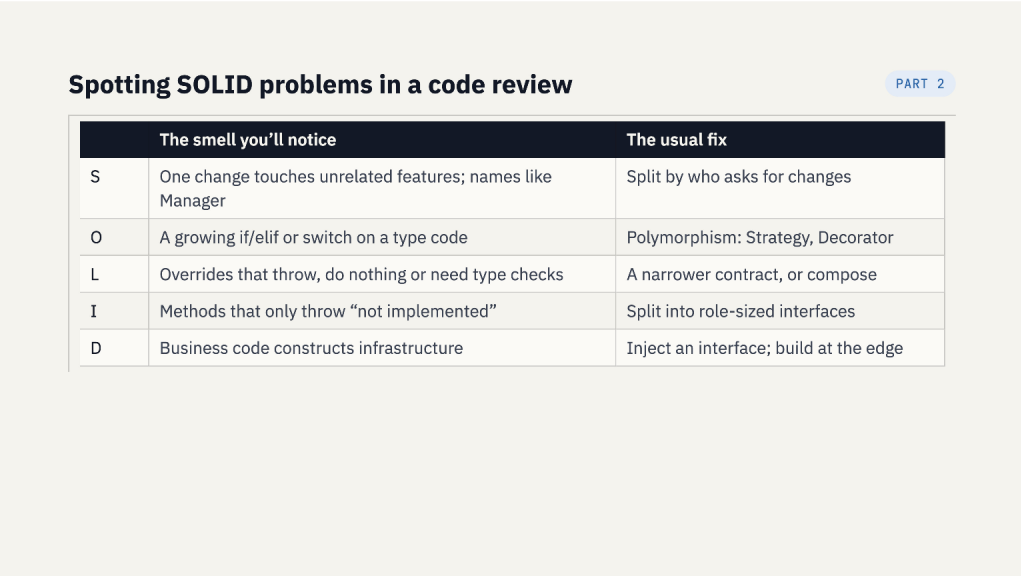
Split the interface by role. VeggieKitchen implements BaseIngredients, a meat kitchen implements both, and cheese_pizza asks only for what it uses.



The business logic constructs its own database class. To test place() you need a database, and to change storage you edit business code.



OrderService asks for anything that satisfies OrderRepository, and the caller decides: an in-memory fake in tests, Postgres in production. For the Java developers: this is exactly what Spring does with constructor injection. Build concrete objects at the edge of the app, and pass them in.

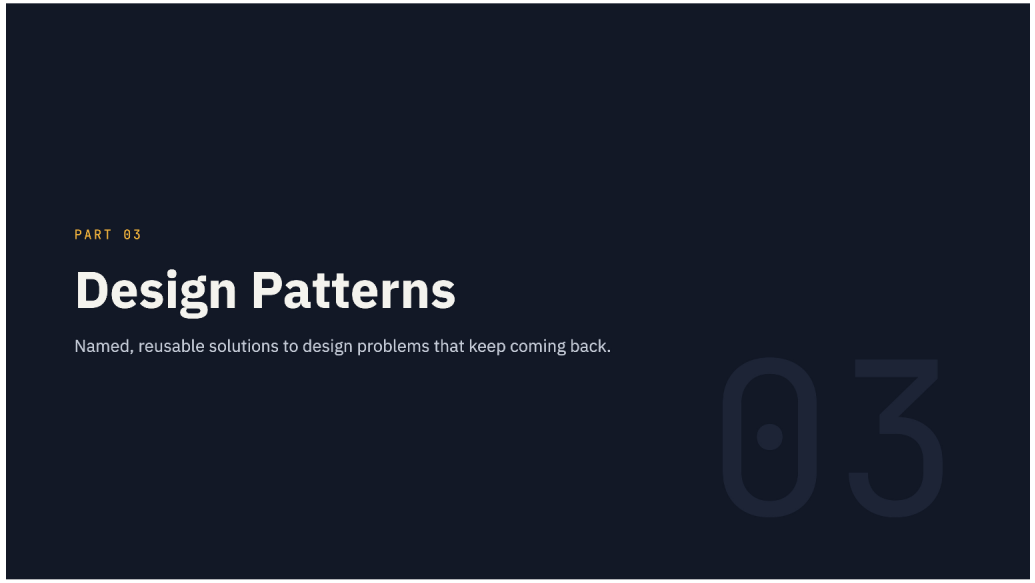


Spotting SOLID problems in a code review

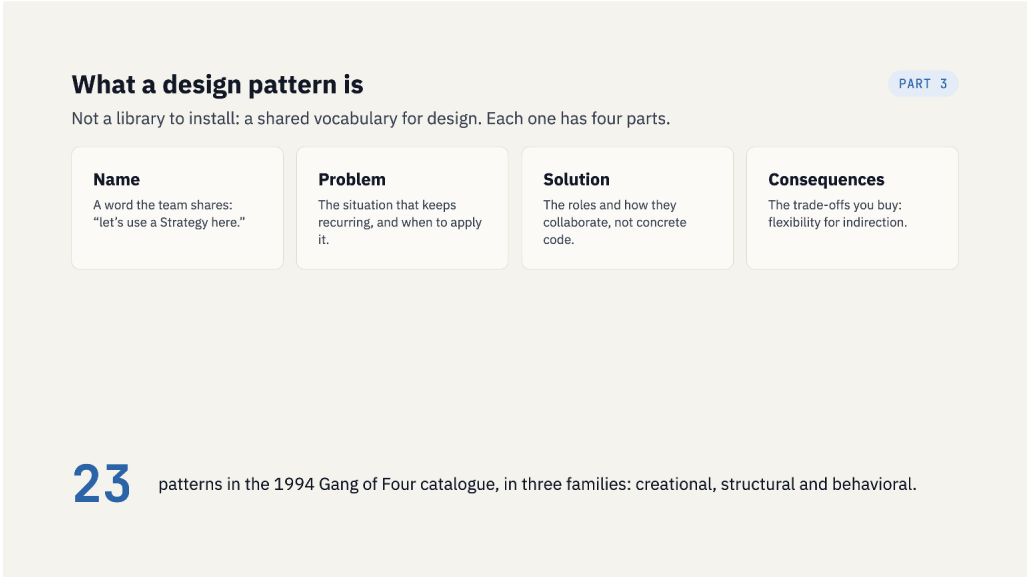
PART 2

	The smell you'll notice	The usual fix
S	One change touches unrelated features; names like Manager	Split by who asks for changes
O	A growing if/elif or switch on a type code	Polymorphism: Strategy, Decorator
L	Overrides that throw, do nothing or need type checks	A narrower contract, or compose
I	Methods that only throw "not implemented"	Split into role-sized interfaces
D	Business code constructs infrastructure	Inject an interface; build at the edge

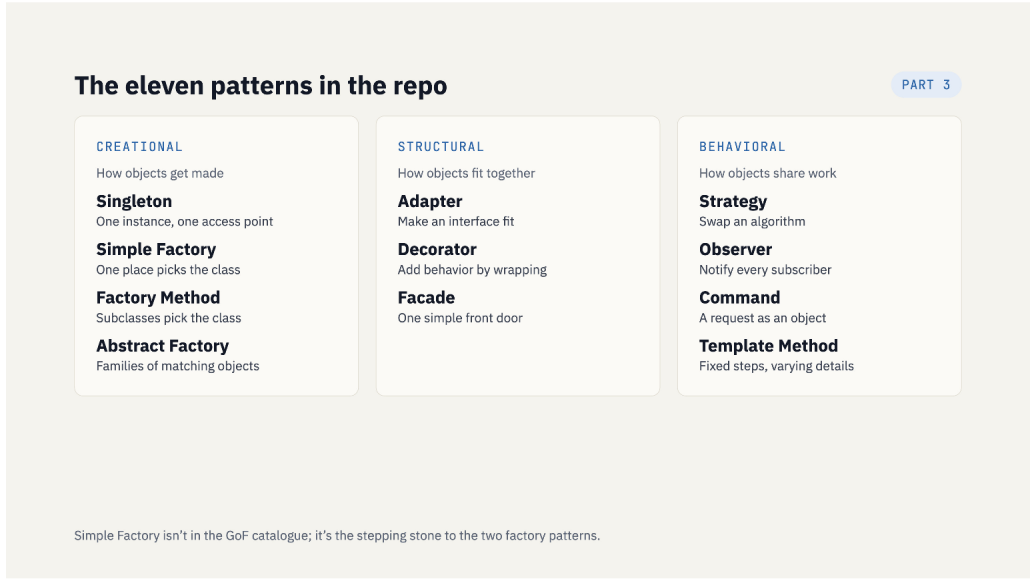
This is the slide to keep. In a code review you rarely say 'this violates OCP'; you notice a smell. Each row maps a smell you can see in a diff, in either language, to the principle and the usual fix.



Part 3: the patterns. We’ll go through eleven, the ones the repo implements, in its three families: creational, structural and behavioral.



A pattern is not code you copy. It’s a name, a problem, a solution described as roles, and the consequences. That’s why the same pattern looks almost identical in Python and Java on the next slides. The Gang of Four book catalogued 23 of them in 1994.



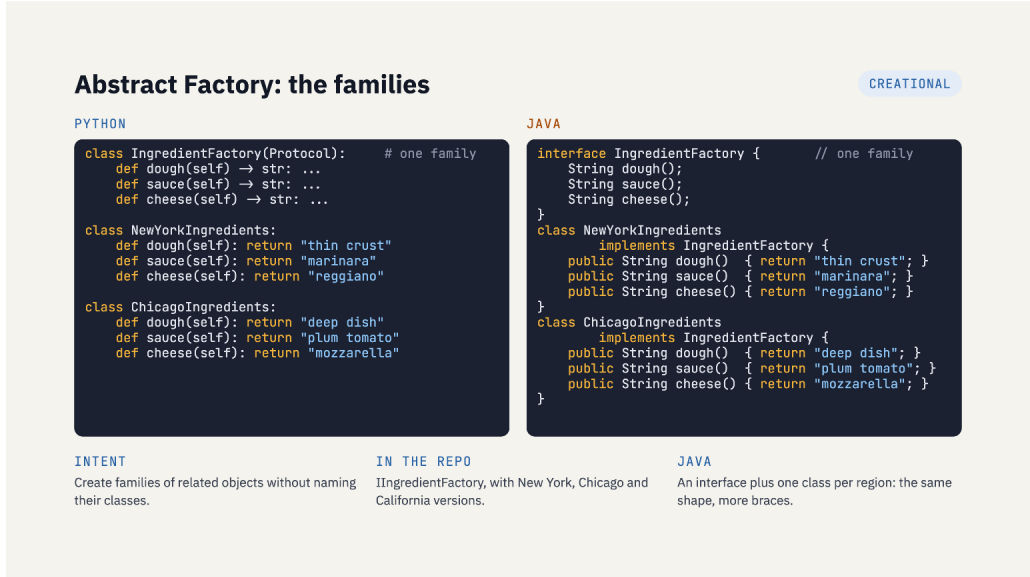
Here’s the map for the rest of the lecture. Creational patterns are about how objects get created, structural ones about how objects are wrapped and combined, behavioral ones about how responsibilities are split. Several patterns take two slides, because Python and Java side by side need the room.

The repo has two C# versions: a static initializer, and this double-checked lock. Two threads can't both create the instance, because the second check runs inside the lock. Java needs volatile so that other threads see a fully constructed object.

In practice neither language needs the hand-written lock. Python: a cached function, or just a module, since modules are imported once. Java: the lazy holder idiom, or an enum with one value, which Effective Java recommends. The bigger point: a singleton is global state. Prefer to create one instance at startup and pass it in.

The simplest creational idea: put the ‘which class?’ decision in one place. The repo uses a switch expression. In Python a dict from enum to class is enough, because calling a class creates an instance; Java gets the same with constructor references and a Supplier. A Java switch on the enum would also work, and the compiler checks that every case is covered.

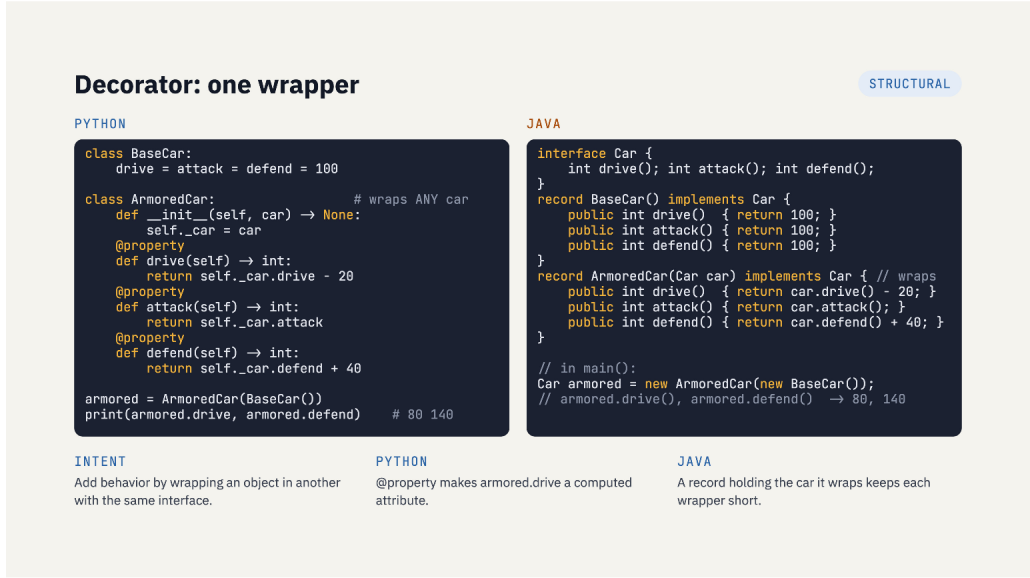
order_pizza is the same for every store: create, prepare, bake, cut, box. Only the creation step differs, so it’s an abstract method that each store overrides. That overridable creation step is the factory method, and it’s one of the good uses of inheritance from Part 1.



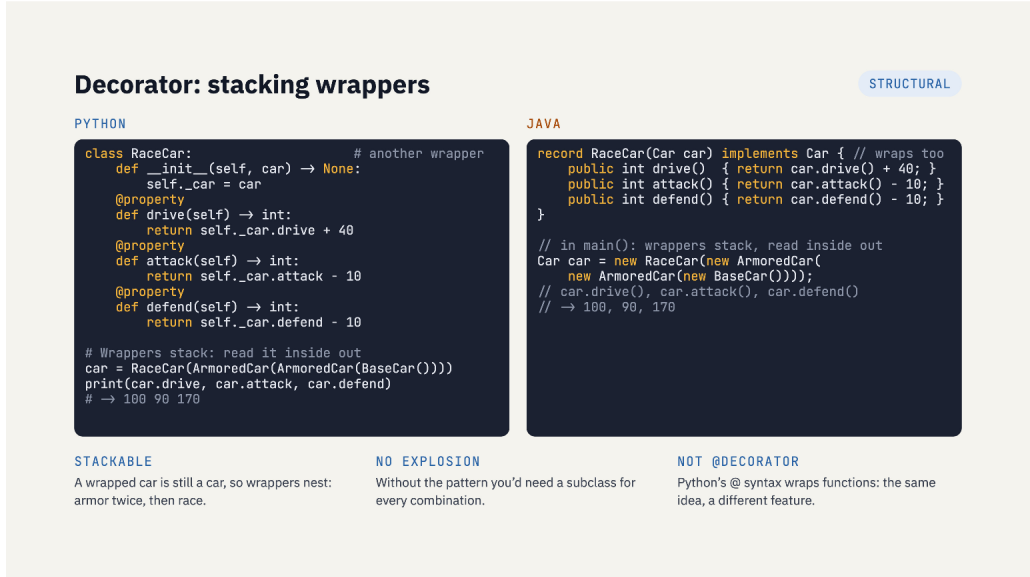
Factory Method creates one product; Abstract Factory creates a family of products that must match: dough, sauce and cheese from the same region.

The pizza doesn't know which region it's in; it just asks its ingredient factory. Swap the factory and you get a consistent set. The factory is injected, so this is composition, not inheritance.

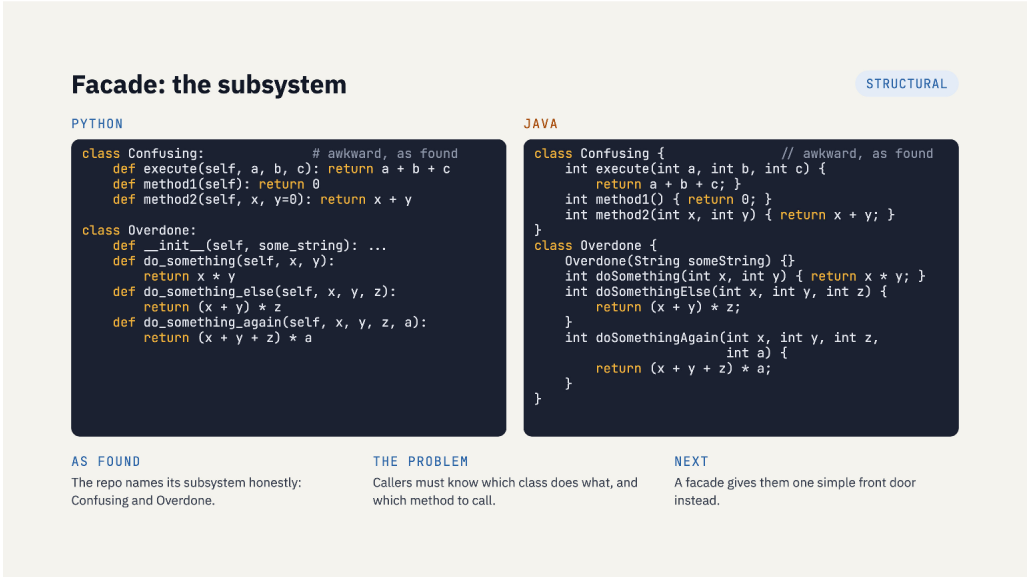
The game only knows Character: chase, flee, attack. A Moose has run and charge, and we can’t or don’t want to change it. The adapter wraps a Moose and translates each call. This is how you plug a third-party library into your own interface.



An ArmoredCar looks like a car, holds a car, and adjusts its numbers. The numbers match the repo’s xUnit tests. In Java a record is a neat fit: record ArmoredCar(Car car) is the wrapped car plus the interface methods.



Because a decorated car is still a car, you can stack them: RaceCar of ArmoredCar of ArmoredCar. Without the pattern you'd need a subclass for every combination, the class explosion from Part 1. Don't confuse it with Python's @decorator syntax, which wraps functions at definition time.



Method1, Method2, DoSomethingAgain: the repo’s subsystem is deliberately awkward, like a lot of legacy code.

Facade: the front door

STRUCTURAL

PYTHON

```python
class BetterApi:                    # the facade
    def __init__(self, confusing=None,
                 overdone=None):
        self._c = confusing or Confusing()
        self._o = overdone or Overdone("foo")

    def add_three_numbers(self, a, b, c):
        return self._c.execute(a, b, c)

    def add_then_multiply(self, *nums, factor):
        calls = {1: self._o.do_something,
                 2: self._o.do_something_else,
                 3: self._o.do_something_again}
        return calls[len(nums)](*nums, factor)

api = BetterApi()
api.add_then_multiply(4, 3, factor=5)   # 35
```

JAVA

```java
class BetterApi {                   // the facade
    private final Confusing c = new Confusing();
    private final Overdone o = new Overdone("foo");

    int addThreeNumbers(int a, int b, int x) {
        return c.execute(a, b, x);
    }
    // Java has real overloads, like the repo's C#:
    int addThenMultiply(int a, int f) {
        return o.doSomething(a, f);
    }
    int addThenMultiply(int a, int b, int f) {
        return o.doSomethingElse(a, b, f);
    }
    int addThenMultiply(int x, int y, int z, int f) {
        return o.doSomethingAgain(x, y, z, f);
    }
}
```

INTENT

Callers see two well-named methods and never touch the subsystem.

PYTHON

No overloads, so *nums and a dict pick the right call.

JAVA

Real overloads, as in the repo's C#: one method per argument count.

The facade gives callers two well-named methods and hides which class does what. A facade doesn’t add behavior; it reduces what callers need to know. Python has no overloads, so it takes *nums and picks the method with a dict; Java keeps the repo’s three overloads.

Strategy: the powers

BEHAVIORAL

PYTHON

```python
class SuperPower(Protocol):
    def exercise_power(self) -> str: ...

class Fly:
    def exercise_power(self) -> str:
        return "Up, Up, and Away!"

class WeaveWeb:
    def exercise_power(self) -> str:
        return "Catches Thieves Like Flies!"

# Each power is a small, swappable object.
# Python could also just pass a function.
```

JAVA

```java
interface SuperPower {
    String exercisePower();
}

class Fly implements SuperPower {
    public String exercisePower() {
        return "Up, Up, and Away!";
    }
}

class WeaveWeb implements SuperPower {
    public String exercisePower() {
        return "Catches Thieves Like Flies!";
    }
}

// A one-method interface also takes a lambda:
SuperPower fight = () -> "KaBaam!";
```

INTENT

Put each algorithm behind one interface and make them swappable.

PYTHON

A Protocol; each power is a small class with one method.

JAVA

An interface; with one method, a lambda can be a power too.

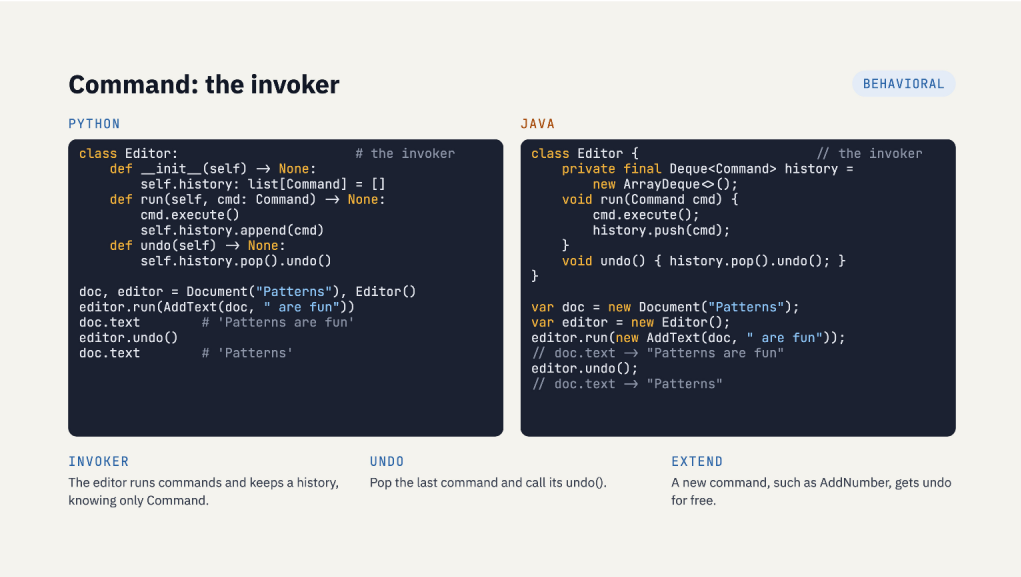
This is the repo’s Heroes example, and the composition fix from Part 1 with a name. First, the strategies: each super power is one small class behind one interface.

Hero delegates to whatever SuperPower it holds. SpiderMan starts with WeaveWeb, and change_super_power swaps it for Fly at runtime. The repo also gives every hero a RegularJoe power by default. Hero depends on the interface, never on a concrete power: DIP and OCP together.

The repo’s SportsAggregator is the subject. Newspapers and radio stations register, and every new game result is pushed to all of them. The aggregator knows nothing about them except that they have update().

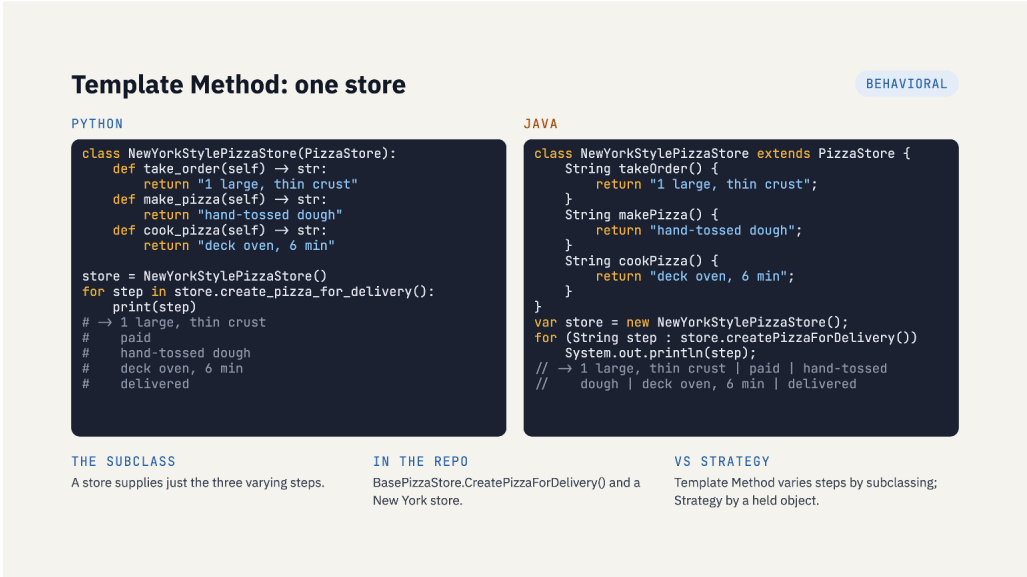
Any object with update() can subscribe. The repo also has a weak-reference version so forgotten observers don’t leak; in Python that’s weakref.WeakSet, in Java a WeakReference or WeakHashMap.

The repo’s Command example adds numbers and text and can remove the last entry. Here it’s in the classic GoF shape: each request is an object with execute and undo.



Because the editor only knows the Command interface, undo works for any command we add later, like an AddNumber command, which is one of the lab exercises.

create_pizza_for_delivery is the template: take order, pay, make, cook, deliver, always in that order. Payment and delivery are the same for every store, so they live in the base; the three varying steps are abstract. This is the inheritance success story from Part 1.



The New York store fills in three methods and inherits everything else. Compare it with Strategy: Template Method varies steps through subclassing, Strategy through a held object.

How the patterns reuse behavior

PUTTING IT TOGETHER

Pattern	Reuses behavior by	Serves
Strategy	composition: holds a swappable object	OCP, DIP
Decorator	composition: wraps the same interface	OCP, SRP
Adapter	composition: wraps a foreign interface	DIP, ISP
Facade	composition: hides a subsystem	SRP, DIP
Observer	composition: a list of subscribers	OCP, DIP
Command	composition: requests as objects	SRP, OCP
Abstract Factory	composition: an injected product family	DIP, OCP
Factory & Template Method	inheritance: a subclass fills a hook	OCP
Singleton	neither: one global instance	works against DIP

This ties the three parts together. Seven of the ten GoF patterns we saw are built on composition, two on a careful, shallow use of inheritance, and Singleton is the odd one out. Each pattern exists to serve one or two SOLID principles: patterns are just principles applied to recurring problems, in any language.

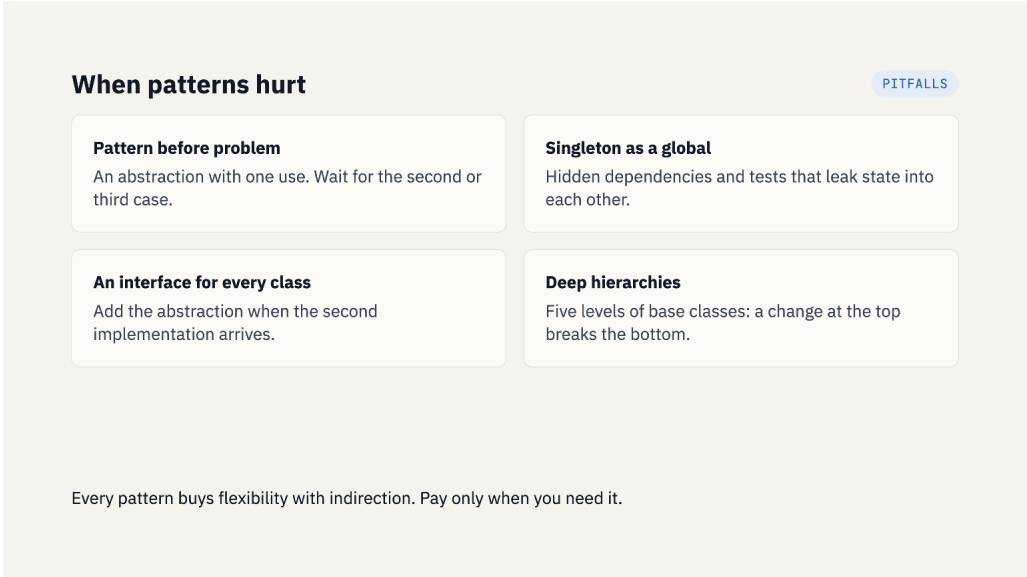
When the language makes a pattern disappear

SHORTCUTS

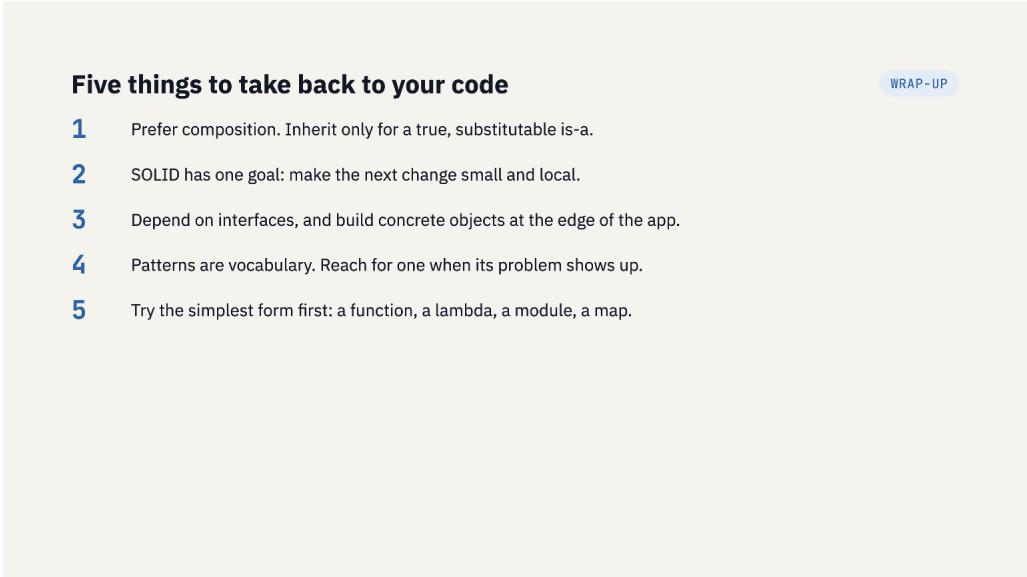
Pattern	In Python	In modern Java	Keep the class when…
Strategy	pass a function	a lambda	the strategy has state or several methods
Command	a callable, functools.partial	a Runnable lambda	you need undo, history or a queue
Singleton	a module, or @functools.cache	an enum	almost never: inject it instead
Observer	a list of callbacks	a list of Consumer<T>	observers need several hooks
Simple Factory	a dict of classes	Map<K, Supplier<T>>	creation needs real logic
Iterator	a generator with yield	Iterable, Stream	rarely

Peter Norvig (1996): 16 of the 23 GoF patterns become simpler or invisible in dynamic languages.

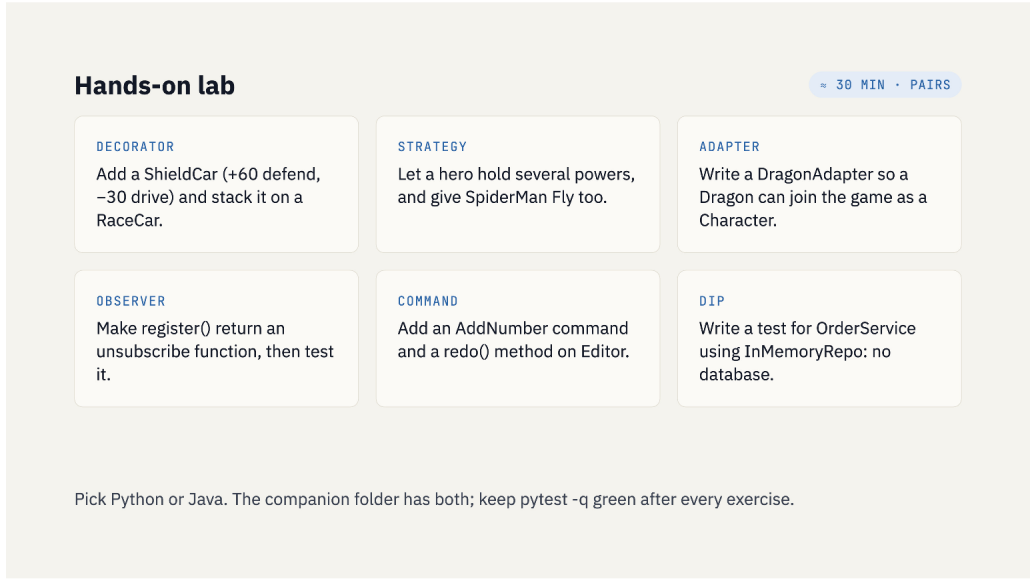
Many patterns exist to work around missing language features. Python has first-class functions, so a one-method strategy can just be a function. Since Java 8, lambdas do the same for any one-method interface. Modules and enums are singletons. Generators and streams are the Iterator pattern built in. Start with the simplest form, and promote it to a class when it needs state, several methods or undo.



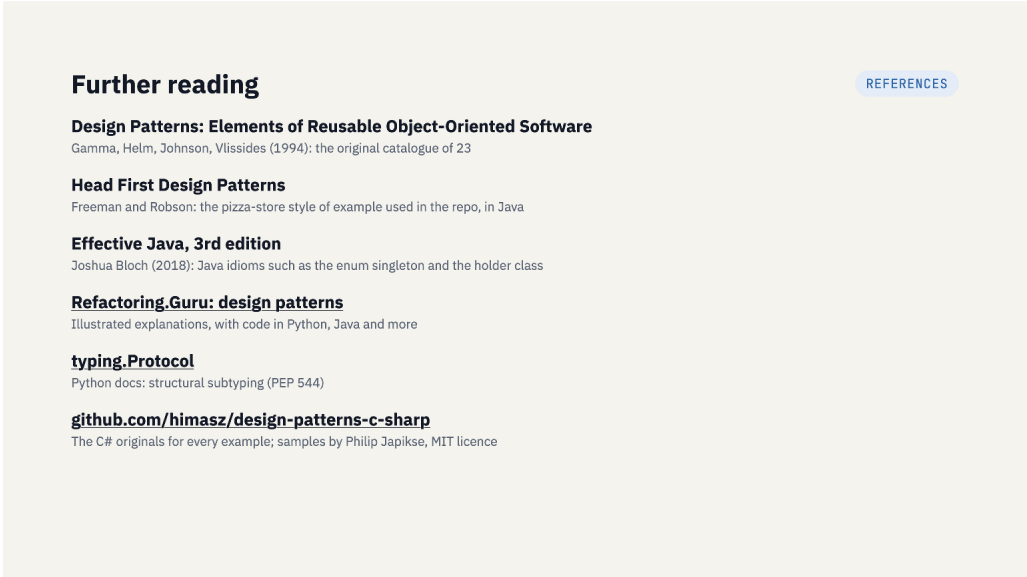
Patterns have a cost: more classes, more indirection, more places to look. The most common mistake is applying a pattern because we just learned it. A good rule is the rule of three: tolerate duplication once, refactor to an abstraction when the third case arrives. An interface with exactly one implementation, in either language, is usually just noise.



If you remember nothing else, remember these five. The first three apply in any language, as the Java next to every Python example showed; the last two are about using patterns with judgment.



Six short exercises, one per pattern or principle, each around five to ten minutes. Work in pairs, in either language: the companion folder has every slide's code in both. A good question to discuss afterwards: did you need to edit any existing class, or only add new ones? That's Open/Closed in practice.



If you read one book, make it Head First Design Patterns; it’s the most approachable, its examples are in Java, and the repo’s pizza stores come from that tradition. The GoF book is the reference, Effective Java covers the Java idioms on the Singleton slide, and Robert C. Martin’s Clean Architecture is the SOLID reference. The original C# samples are linked here, with credit to Philip Japikse.

Thanks. Open the floor for questions. Good prompts if it’s quiet: Where in our own codebase is there a growing if/elif or switch we could turn into a Strategy? Which of our classes would fail the ‘describe it without and’ test?